

MAWLANA BHASHANI SCIENCE AND TECHNOLOGY UNIVERSITY

SANTOSH, TANGAIL-1902



DEPARTMENT OF INFORMATION AND COMMUNICATION TECHNOLOGY

Lab Report

Lab Report No : 06

Lab Report on : Design and Deploy a Tasbih App or Mantra Counting App by using
JavaScript.

Course Title : Web Application Development Lab

Course Code : ICT-3204

Submitted By	Submitted To
Name: Kuldip Saha Mugdha ID: IT-22018 3 rd Year, 2 nd Semester Session: 2021-2022 Dept. of ICT, MBSTU	Mr. Md. Anowar Kabir Lecturer DEPARTMENT OF INFORMATION AND COMMUNICATION TECHNOLOGY MAWLANA BHASHANI SCIENCE AND TECHNOLOGY UNIVERSITY

Date of Performance: 12/01/26

Date of Submission: 16/01/26

Introduction:

A Mantra Counting App is a digital alternative to traditional prayer beads. In many spiritual practices, especially in Hinduism and Buddhism, a mantra is recited **108 times**, which is considered spiritually significant.

JavaScript enables dynamic behavior in web applications, such as:

- Handling button click events
- Updating counters in real time
- Applying logic conditions (minimum and maximum limits)
- Providing visual feedback using CSS animations

This experiment demonstrates how JavaScript can be combined with HTML and CSS to create an interactive, user-friendly spiritual utility application.

Objective:

The objective of this experiment is to design and develop a simple, interactive Tasbih (Mantra Counting) web application using **HTML**, **CSS**, and **JavaScript**. The application allows users to count mantras digitally, reset the counter, decrease counts, and track completed spiritual goals based on a fixed count of 108.

Tools & Technologies Used:

- **HTML5** – Structure of the web page
- **CSS3** – Styling and animations
- **JavaScript** – Application logic and interactivity
- **Web Browser** – Google Chrome
- **Code Editor** – VS Code

System Requirements:

- Operating System: Windows
- Browser: Any modern web browser
- Hardware: Standard Laptop

System Design:

The application consists of three main components:

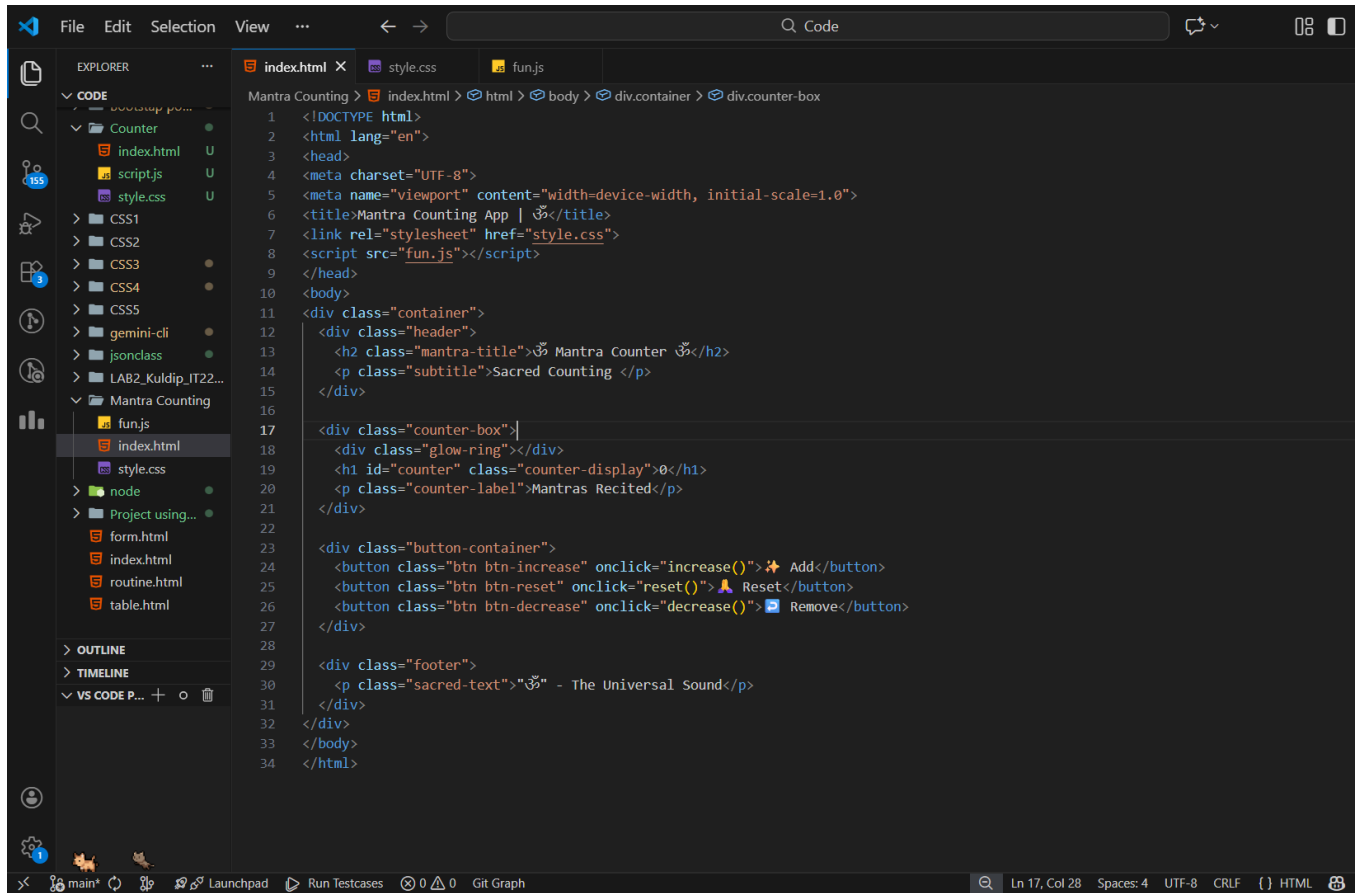
1. **User Interface (HTML + CSS)**
 - Counter display
 - Buttons (Add, Remove, Reset)
 - Spiritual goal tracker
2. **Application Logic (JavaScript)**
 - Increment and decrement counter
 - Reset counter
 - Detect completion of 108 counts
 - Update completed spiritual goals
3. **Visual Feedback System**
 - Glow animations
 - Pulse effect when a goal is completed

Algorithm:

1. Initialize count = 0 and spiritualGoals = 0
2. When **Add** button is clicked:
 - Increase count by 1
 - If count reaches 108:
 - Increment spiritual goal
 - Reset count to 0
 - Show animation
3. When **Remove** button is clicked:
 - Decrease count by 1 (minimum 0)
4. When **Reset** button is clicked:
 - Set count to 0
5. Update display after every operation

Program Code:

HTML Code:

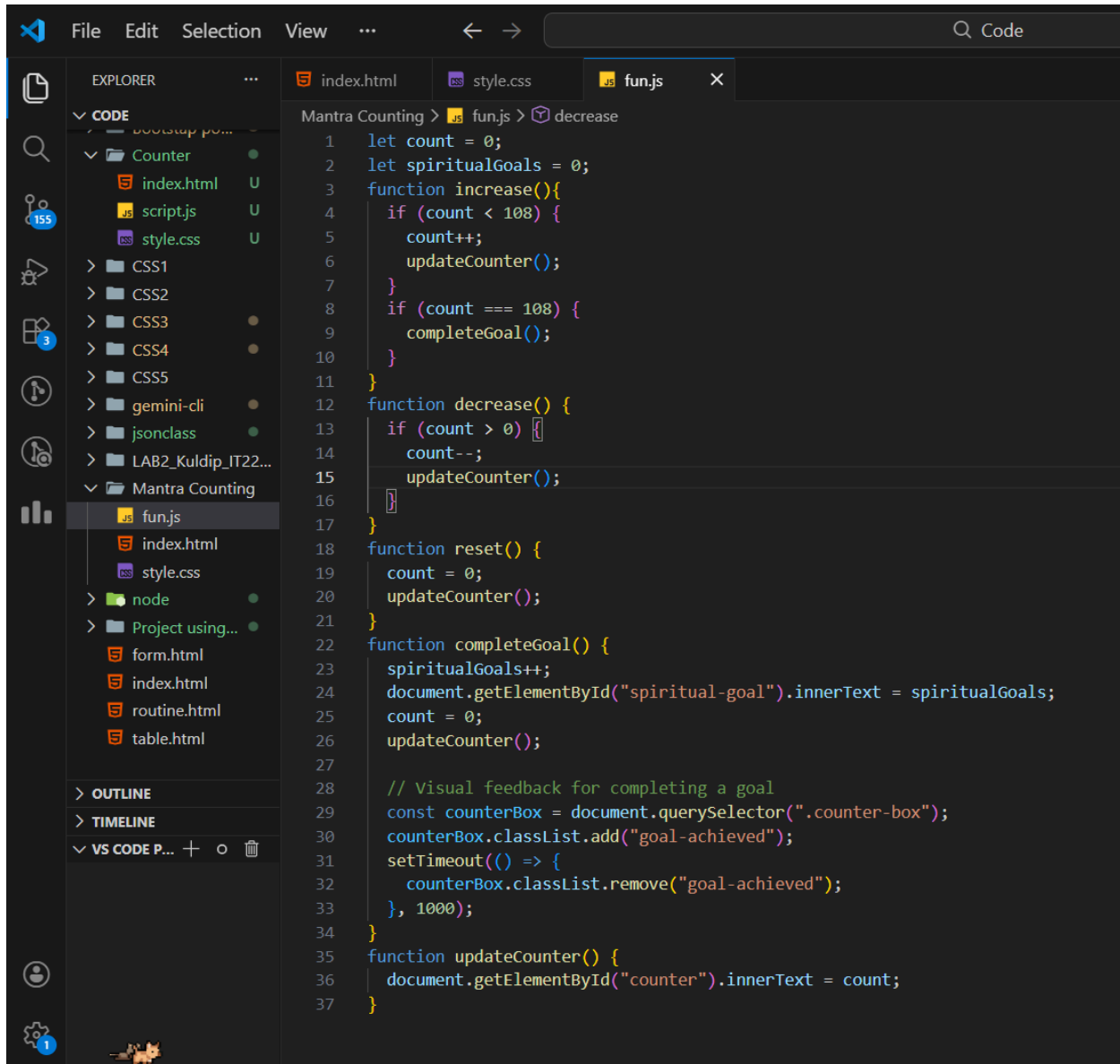


The screenshot shows the Visual Studio Code editor with a project named "Mantra Counting". The Explorer sidebar on the left shows the file structure, including a "Counter" folder with "index.html", "style.css", and "fun.js". The main editor area displays the "index.html" file, which contains the following HTML code:

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <meta charset="UTF-8">
5 <meta name="viewport" content="width=device-width, initial-scale=1.0">
6 <title>Mantra Counting App | ॐ</title>
7 <link rel="stylesheet" href="style.css">
8 <script src="fun.js"></script>
9 </head>
10 <body>
11 <div class="container">
12 <div class="header">
13 <h2 class="mantra-title">ॐ Mantra Counter ॐ</h2>
14 <p class="subtitle">Sacred Counting </p>
15 </div>
16
17 <div class="counter-box">
18 <div class="glow-ring"></div>
19 <h1 id="counter" class="counter-display">0</h1>
20 <p class="counter-label">Mantras Recited</p>
21 </div>
22
23 <div class="button-container">
24 <button class="btn btn-increase" onclick="increase()">✚ Add</button>
25 <button class="btn btn-reset" onclick="reset()">🔄 Reset</button>
26 <button class="btn btn-decrease" onclick="decrease()">✚ Remove</button>
27 </div>
28
29 <div class="footer">
30 <p class="sacred-text">"ॐ" - The Universal Sound</p>
31 </div>
32 </div>
33 </body>
34 </html>
```

The status bar at the bottom indicates the current position is Line 17, Column 28, with 4 spaces, UTF-8 encoding, and CRLF line endings. The file type is identified as HTML.

JavaScript Code:

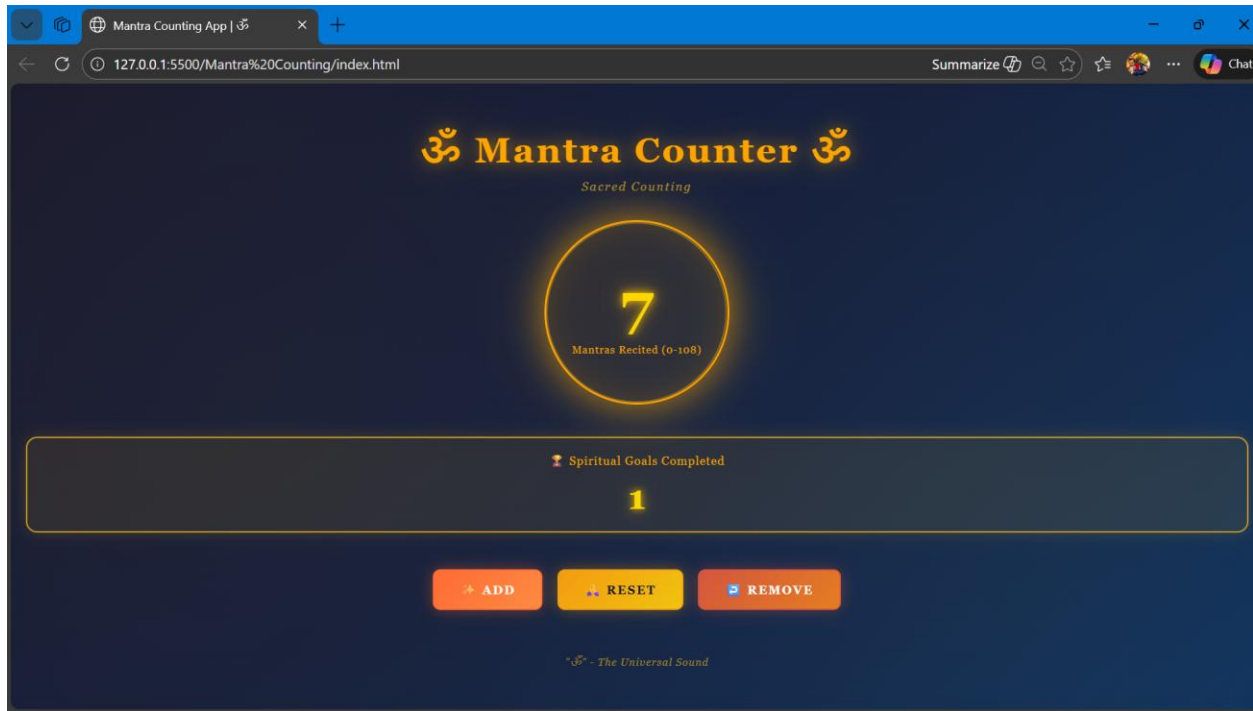


The screenshot shows the Visual Studio Code editor interface. The Explorer panel on the left displays a project structure with folders like 'CSS1' through 'CSS5', 'gemini-cli', 'jsonclass', 'LAB2_Kuldip_IT22...', and 'Mantra Counting'. The 'Mantra Counting' folder is expanded, showing files 'fun.js', 'index.html', and 'style.css'. The 'fun.js' file is selected and its content is displayed in the main editor area. The code defines variables for 'count' and 'spiritualGoals', and implements functions for 'increase()', 'decrease()', 'reset()', 'completeGoal()', and 'updateCounter()'. The 'completeGoal()' function includes a timeout to remove a 'goal-achieved' class from a 'counter-box' after 1000ms. The 'updateCounter()' function updates the text of a 'counter' element.

```
1 let count = 0;
2 let spiritualGoals = 0;
3 function increase(){
4     if (count < 108) {
5         count++;
6         updateCounter();
7     }
8     if (count === 108) {
9         completeGoal();
10    }
11 }
12 function decrease() {
13     if (count > 0) {
14         count--;
15         updateCounter();
16     }
17 }
18 function reset() {
19     count = 0;
20     updateCounter();
21 }
22 function completeGoal() {
23     spiritualGoals++;
24     document.getElementById("spiritual-goal").innerText = spiritualGoals;
25     count = 0;
26     updateCounter();
27
28     // Visual feedback for completing a goal
29     const counterBox = document.querySelector(".counter-box");
30     counterBox.classList.add("goal-achieved");
31     setTimeout(() => {
32         counterBox.classList.remove("goal-achieved");
33     }, 1000);
34 }
35 function updateCounter() {
36     document.getElementById("counter").innerText = count;
37 }
```

Output:

- Counter increases and decreases correctly
- Maximum count limited to 108
- On reaching 108, a spiritual goal is completed
- Counter resets automatically after goal completion
- Attractive UI with animations and glow effects



Result:

The Mantra Counting App was successfully designed and implemented using JavaScript. The application performs accurate counting, tracks completed spiritual goals, and provides a smooth and visually engaging user experience.

Live link of GitHub: [Mugdha67's Mantra Counting App](#)

Discussion:

This experiment demonstrates how JavaScript interacts with HTML elements to create dynamic behavior in web applications. The use of CSS animations enhances user experience, while JavaScript functions handle the core logic efficiently. The project helps in understanding event-driven programming and real-time UI updates.

Conclusion:

The Mantra Counting App was successfully implemented using HTML, CSS, and JavaScript. This experiment improved understanding of JavaScript functions, event handling, and DOM manipulation. Such applications can be extended further by adding features like sound effects, local storage, or daily count tracking.